

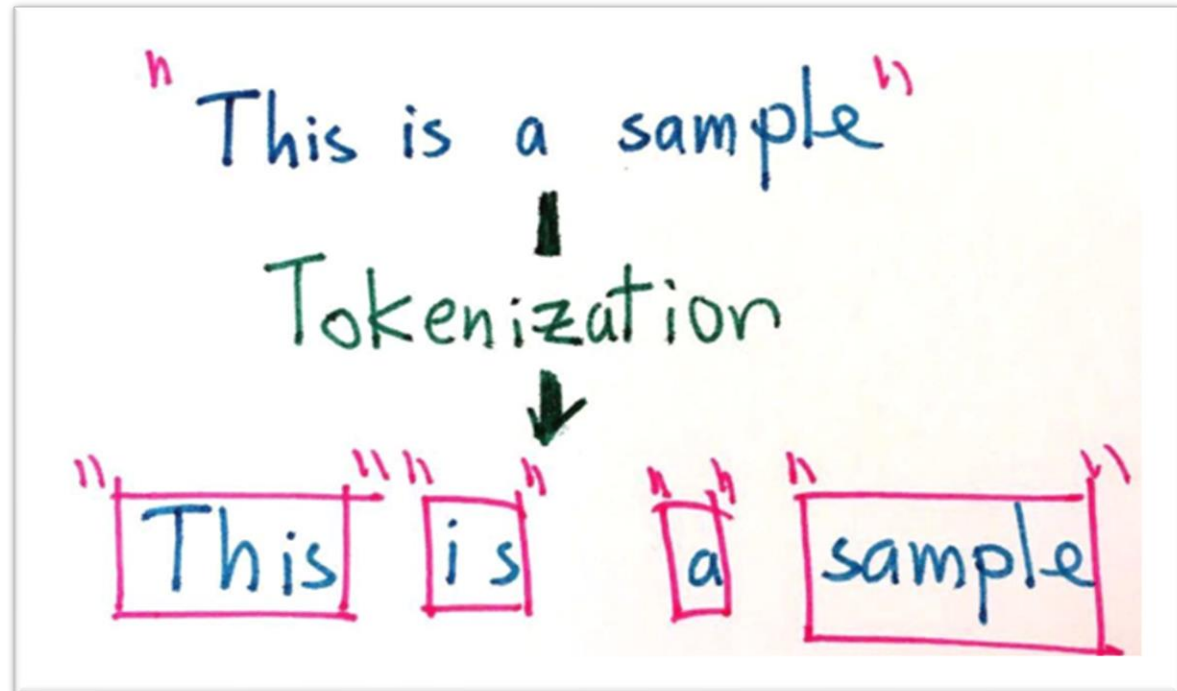
TOKENIZATION IN NLP



Tokenization in Natural Language Processing (NLP)

Tokenization is the process of converting a text into smaller, meaningful units called tokens, such as words, subwords, or characters, to prepare the text for further analysis in natural language processing tasks. Tokenization is a critical step in many NLP tasks, including text processing, language modelling, and machine translation.

The process involves splitting a string, or text into a list of tokens. One can think of tokens as parts like a word is a token in a sentence, and a sentence is a token in a paragraph.



The Importance of Tokenization

Tokenization is a crucial step in text processing and natural language processing (NLP) for several reasons.

- Effective Text Processing:** Tokenization reduces the size of raw text so that it can be handled more easily for processing and analysis.
- Feature extraction:** Text data can be represented numerically for algorithmic comprehension by using tokens as features in machine learning models.
- Language Modelling:** Tokenization in NLP facilitates the creation of organized representations of language, which is useful for tasks like text generation and language modelling.
- Information Retrieval:** Tokenization is essential for indexing and searching in systems that store and retrieve information efficiently based on words or phrases.
- Text Analysis:** Tokenization is used in many NLP tasks, including sentiment analysis and named entity recognition, to determine the function and context of individual words in a sentence.
- Vocabulary Management:** By generating a list of distinct tokens that stand in for words in the dataset, tokenization helps manage a corpus's vocabulary.
- Task-Specific Adaptation:** Tokenization can be customized to meet the needs of particular NLP tasks, meaning that it will work best in applications such as summarization and machine translation.
- Preprocessing Step:** This essential preprocessing step transforms unprocessed text into a format appropriate for additional statistical and computational analysis.

Types of Tokenization

Tokenization can be classified into several types based on how the text is segmented. Here are some types of tokenization:

- **Word Tokenization**
- **Sentence Tokenization**
- **Subword Tokenization**
- **Character Tokenization**

Word Tokenization:

Word tokenization divides the text into individual words.

Splits text into individual words based on spaces and punctuation.

Example:

```
Input: "Tokenization is an important NLP task."
```

```
Output: ["Tokenization", "is", "an", "important", "NLP", "task", "."]
```

Sentence Tokenization:

Definition: Splits text into individual sentences, usually based on punctuation such as periods, exclamation marks, or question marks.

Example:

```
Input: "Tokenization is an important NLP task. It helps break down text into smaller units."  
Output: ["Tokenization is an important NLP task.", "It helps break down text into smaller units."]
```

Subword Tokenization:

Subword tokenization entails breaking down words into smaller units (subwords), which can be especially useful when dealing with morphologically rich languages or rare words, unknown words.

Example:

```
Input: "tokenization"  
Output: ["token", "ization"]
```

Character Tokenization:

This process divides the text into individual characters. This can be useful for modelling character-level language.

Example:

```
Input: "Tokenization"  
Output: ["T", "o", "k", "e", "n", "i", "z", "a", "t", "i", "o", "n"]
```

Implementation for Tokenization

❑ Sentence Tokenization using sent_tokenize :-

```
# Importing necessary libraries
import nltk
from nltk.tokenize import sent_tokenize

# Sample text for tokenization
text = "I love NLP. It's fascinating! Sentence tokenization is very useful."

# Sentence Tokenization
sentences = sent_tokenize(text)

# Output the tokenized sentences
print(sentences)
```

```
['I love NLP.', "It's fascinating!", 'Sentence tokenization is very useful.']
```

❑ Word Tokenization using word_tokenize :-

```
# Importing necessary libraries  
from nltk.tokenize import word_tokenize
```

```
# Sample text for tokenization  
text = "Tokenization is the process of breaking down text into tokens."
```

```
# Word Tokenization  
words = word_tokenize(text)
```

```
# Output the tokenized words  
print(words)
```

```
['Tokenization', 'is', 'the', 'process', 'of', 'breaking', 'down', 'text', 'into', 'tokens', '.']
```

❏ Character Tokenization :-

```
# Importing necessary libraries
```

```
import nltk
```

```
# Sample text for character tokenization
```

```
text = "Character tokenization."
```

```
# Character Tokenization
```

```
characters = list(text)
```

```
# Output the tokenized characters
```

```
print(characters)
```

```
['C', 'h', 'a', 'r', 'a', 'c', 't', 'e', 'r', ' ', 't', 'o', 'k', 'e', 'n', 'i', 'z', 'a', 't', 'i', 'o', 'n', '.']
```

```
from nltk.tokenize import word_tokenize
```

```
# Sample text
```

```
text = "Character tokenization is easy with NLTK."
```

```
# First, apply word tokenization
```

```
words = word_tokenize(text)
```

```
# Then, apply character tokenization to each word
```

```
character_tokens = [list(word) for word in words]
```

```
# Output the tokenized characters for each word
```

```
print(character_tokens)
```

```
[['C', 'h', 'a', 'r', 'a', 'c', 't', 'e', 'r'], ['t', 'o', 'k', 'e', 'n', 'i', 'z', 'a', 't', 'i', 'o', 'n'], ['i', 's'], ['e', 'a', 's', 'y'], ['w', 'i', 't', 'h'], ['N', 'L', 'T', 'K'], ['.']]
```


Swami Vivekananda, born Narendranath Datta in 1863 in Calcutta, India, was a profound spiritual leader, philosopher, and social reformer who played a pivotal role in the revitalization of Hinduism and the spread of Indian spiritual wisdom to the Western world. His life and teachings continue to inspire millions of people worldwide.

Vivekananda was deeply influenced by his guru, Sri Ramakrishna Paramahansa, and their profound spiritual connection shaped his mission in life. He was a proponent of Vedanta, emphasizing the oneness of all religions and the universality of spiritual truths. In 1893, he represented Hinduism at the Parliament of the World's Religions in Chicago, where his historic speech began with the words, "Sisters and brothers of America," earning him worldwide recognition and acclaim.

Swami Vivekananda's teachings stressed the importance of self-realization, the divinity of the individual soul, and the need for service to humanity. He believed that education was the key to social reform and worked tirelessly to establish the Ramakrishna Mission and Math, institutions that continue to provide education, healthcare, and humanitarian aid.

```
import nltk
```

```
paragraph = """ Swami Vivekananda, born Narendranath Datta in 1863 in Calcutta, India, was a profound spiritual leader, philosopher, and social reformer who played a pivotal role in the revitalization of Hinduism and the spread of Indian spiritual wisdom to the Western world. Vivekananda was deeply influenced by his guru, Sri Ramakrishna Paramahansa, and their profound spiritual connection shaped his mission in life. Swami Vivekananda's teachings stressed the importance of self-realization, the divinity of the individual soul, and the need for service to humanity. He believed that education was the key to social reform and worked tirelessly to establish the Ramakrishna Mission and Math, institutions that continue to provide education, healthcare, and humanitarian aid.
```

```
sentences = nltk.sent_tokenize(paragraph)
```

```
print(sentences)
```

```
[' Swami Vivekananda, born Narendranath Datta in 1863 in Calcutta, India, was a profound spiritual leader, philosopher, and social reformer who played a pivotal role in the revitalization of Hinduism and the spread of Indian spiritual wisdom to the Western world.', 'His life and teachings continue to inspire millions of people worldwide.', 'Vivekananda was deeply influenced by his guru, Sri Ramakrishna Paramahansa, and their profound spiritual connection shaped his mission in life.', 'He was a proponent of Vedanta, emphasizing the oneness of all religions and the universality of spiritual truths.', 'In 1893, he represented Hinduism at the Parliament of the World's Religions in Chicago, where his historic speech began with the words, "Sisters and brothers of America," earning him worldwide recognition and acclaim.', "Swami Vivekananda's teachings stressed the importance of self-realization, the divinity of the individual soul, and the need for service to humanity.", 'He believed that education was the key to social reform and worked tirelessly to establish the Ramakrishna Mission and Math, institutions that continue to provide education, healthcare, and humanitarian aid.']
```

```
words = nltk.word_tokenize(paragraph)
```

```
print(words)
```

```
['Swami', 'Vivekananda', ',', 'born', 'Narendranath', 'Datta', 'in', '1863', 'in', 'Calcutta', ',', 'India', ',', 'was', 'a',  
'profound', 'spiritual', 'leader', ',', 'philosopher', ',', 'and', 'social', 'reformer', 'who', 'played', 'a', 'pivotal', 'rol  
e', 'in', 'the', 'revitalization', 'of', 'Hinduism', 'and', 'the', 'spread', 'of', 'Indian', 'spiritual', 'wisdom', 'to', 'th  
e', 'Western', 'world', '.', 'His', 'life', 'and', 'teachings', 'continue', 'to', 'inspire', 'millions', 'of', 'people', 'world  
wide', '.', 'Vivekananda', 'was', 'deeply', 'influenced', 'by', 'his', 'guru', ',', 'Sri', 'Ramakrishna', 'Paramahansa', ',',  
'and', 'their', 'profound', 'spiritual', 'connection', 'shaped', 'his', 'mission', 'in', 'life', '.', 'He', 'was', 'a', 'propon  
ent', 'of', 'Vedanta', ',', 'emphasizing', 'the', 'oneness', 'of', 'all', 'religions', 'and', 'the', 'universality', 'of', 'spi  
ritual', 'truths', '.', 'In', '1893', ',', 'he', 'represented', 'Hinduism', 'at', 'the', 'Parliament', 'of', 'the', 'World',  
"'s", 'Religions', 'in', 'Chicago', ',', 'where', 'his', 'historic', 'speech', 'began', 'with', 'the', 'words', ',', '``', 'Sis  
ters', 'and', 'brothers', 'of', 'America', ',', '""', 'earning', 'him', 'worldwide', 'recognition', 'and', 'acclaim', '.', 'Swa  
mi', 'Vivekananda', "'s", 'teachings', 'stressed', 'the', 'importance', 'of', 'self-realization', ',', 'the', 'divinity', 'of',  
'the', 'individual', 'soul', ',', 'and', 'the', 'need', 'for', 'service', 'to', 'humanity', '.', 'He', 'believed', 'that', 'edu  
cation', 'was', 'the', 'key', 'to', 'social', 'reform', 'and', 'worked', 'tirelessly', 'to', 'establish', 'the', 'Ramakrishna',  
'Mission', 'and', 'Math', ',', 'institutions', 'that', 'continue', 'to', 'provide', 'education', ',', 'healthcare', ',', 'and',  
'humanitarian', 'aid', '.']
```

```
# Importing necessary libraries
```

```
import nltk
```

```
# Sample paragraph about APJ Abdul Kalam
```

```
text = """Dr. APJ Abdul Kalam was an Indian aerospace scientist and the 11th President of India.
```

```
He is known as the Missile Man of India for his work on the development of ballistic missile technology.
```

```
His life and work inspire millions of people across the world."""
```

```
sentences = sent_tokenize(text)
```

```
print(sentences)
```

```
['Dr. APJ Abdul Kalam was an Indian aerospace scientist and the 11th President of India.', 'He is known as the Missile Man of India for his work on the development of ballistic missile technology.', 'His life and work inspire millions of people across the world.']
```

```
words=word_tokenize(text)
```

```
print(words)
```

```
['Dr.', 'APJ', 'Abdul', 'Kalam', 'was', 'an', 'Indian', 'aerospace', 'scientist', 'and', 'the', '11th', 'President', 'of', 'India', '.', 'He', 'is', 'known', 'as', 'the', 'Missile', 'Man', 'of', 'India', 'for', 'his', 'work', 'on', 'the', 'development', 'of', 'ballistic', 'missile', 'technology', '.', 'His', 'life', 'and', 'work', 'inspire', 'millions', 'of', 'people', 'across', 'the', 'world', '.']
```

```
# Character Tokenization
characters = list(text)
# Output the tokenized characters
print(characters)
```

```
['D', 'r', '.', ' ', 'A', 'P', 'J', ' ', 'A', 'b', 'd', 'u', 'l', ' ', 'K', 'a', 'l', 'a', 'm', ' ', 'w', 'a', 's', ' ', 'a',  
'n', ' ', 'I', 'n', 'd', 'i', 'a', 'n', ' ', 'a', 'e', 'r', 'o', 's', 'p', 'a', 'c', 'e', ' ', 's', 'c', 'i', 'e', 'n', 't',  
'i', 's', 't', ' ', 'a', 'n', 'd', ' ', 't', 'h', 'e', ' ', '1', '1', 't', 'h', ' ', 'P', 'r', 'e', 's', 'i', 'd', 'e', 'n',  
't', ' ', 'o', 'f', ' ', 'I', 'n', 'd', 'i', 'a', '.', '\n', 'H', 'e', ' ', 'i', 's', ' ', 'k', 'n', 'o', 'w', 'n', ' ',  
'a', 's', ' ', 't', 'h', 'e', ' ', 'M', 'i', 's', 's', 'i', 'l', 'e', ' ', 'M', 'a', 'n', ' ', 'o', 'f', ' ', 'I', 'n', 'd',  
'i', 'a', ' ', 'f', 'o', 'r', ' ', 'h', 'i', 's', ' ', 'w', 'o', 'r', 'k', ' ', 'o', 'n', ' ', 't', 'h', 'e', ' ', 'd', 'e',  
'v', 'e', 'l', 'o', 'p', 'm', 'e', 'n', 't', ' ', 'o', 'f', ' ', 'b', 'a', 'l', 'l', 'i', 's', 't', 'i', 'c', ' ', 'm', 'i',  
's', 's', 'i', 'l', 'e', ' ', 't', 'e', 'c', 'h', 'n', 'o', 'l', 'o', 'g', 'y', '.', '\n', 'H', 'i', 's', ' ', 'l', 'i', 'f',  
'e', ' ', 'a', 'n', 'd', ' ', 'w', 'o', 'r', 'k', ' ', 'i', 'n', 's', 'p', 'i', 'r', 'e', ' ', 'm', 'i', 'l', 'l', 'i', 'o',  
'n', 's', ' ', 'o', 'f', ' ', 'p', 'e', 'o', 'p', 'l', 'e', ' ', 'a', 'c', 'r', 'o', 's', 's', ' ', 't', 'h', 'e', ' ', 'w',  
'o', 'r', 'l', 'd', '.']
```